BITS PILANI · M.TECH (AIML) · MACHINE LEARNING

Assignment 2
Chess Endgame Outcome Classification

Dataset: UCI Chess (King-Rook vs. King-Pawn), KRKPA7

Name: Viradiya Tushar Rajeshbhai

BITS ID: 2026aj05073

Prepared: 29 October 2026

1. GitHub repository

https://github.com/tusharviradiya/aiml-ml-classification-assignment-2

Source code, dependencies, test data, five saved pipelines, and evaluation results.

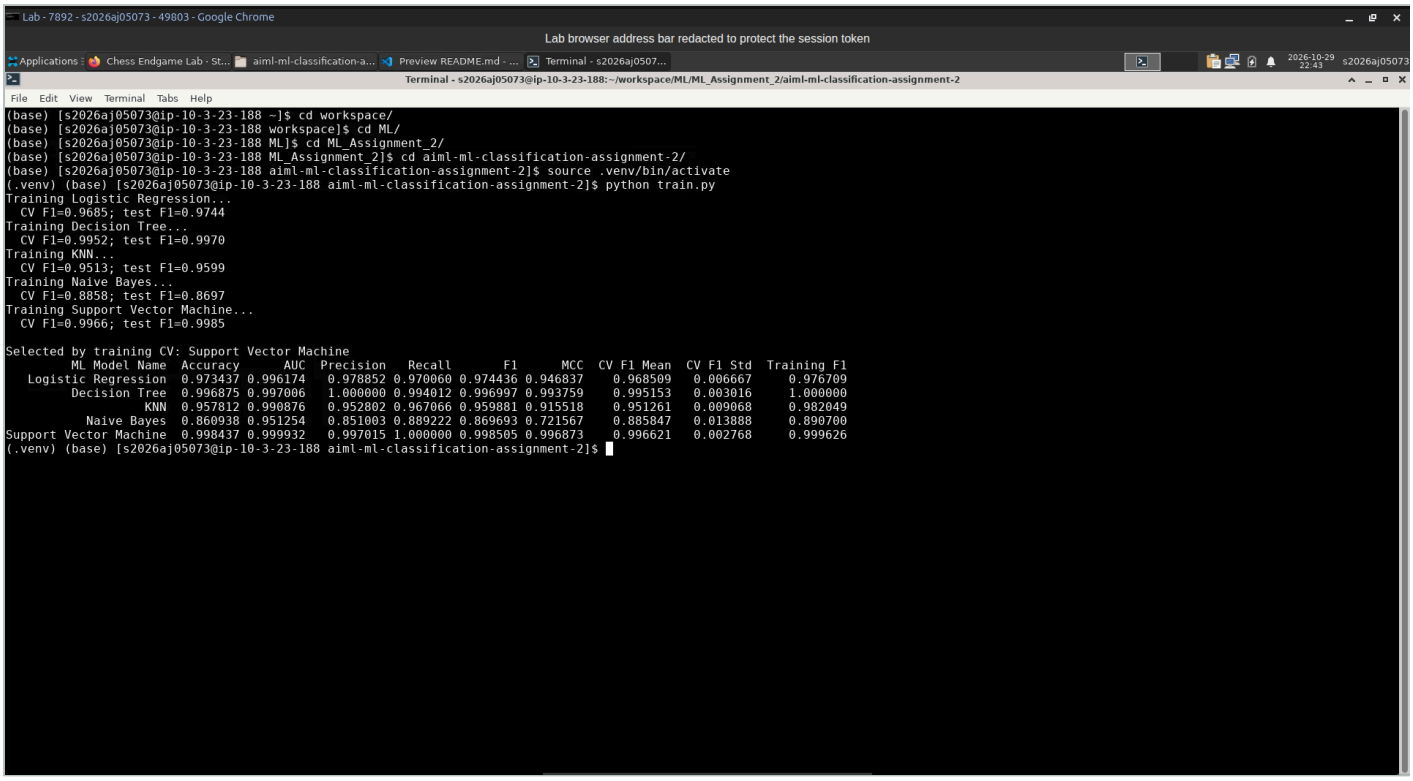
2. Live Streamlit application

https://chess-endgame-lab-tushar.streamlit.app/

Interactive CSV evaluation, model selection, six metrics, and model comparisons.

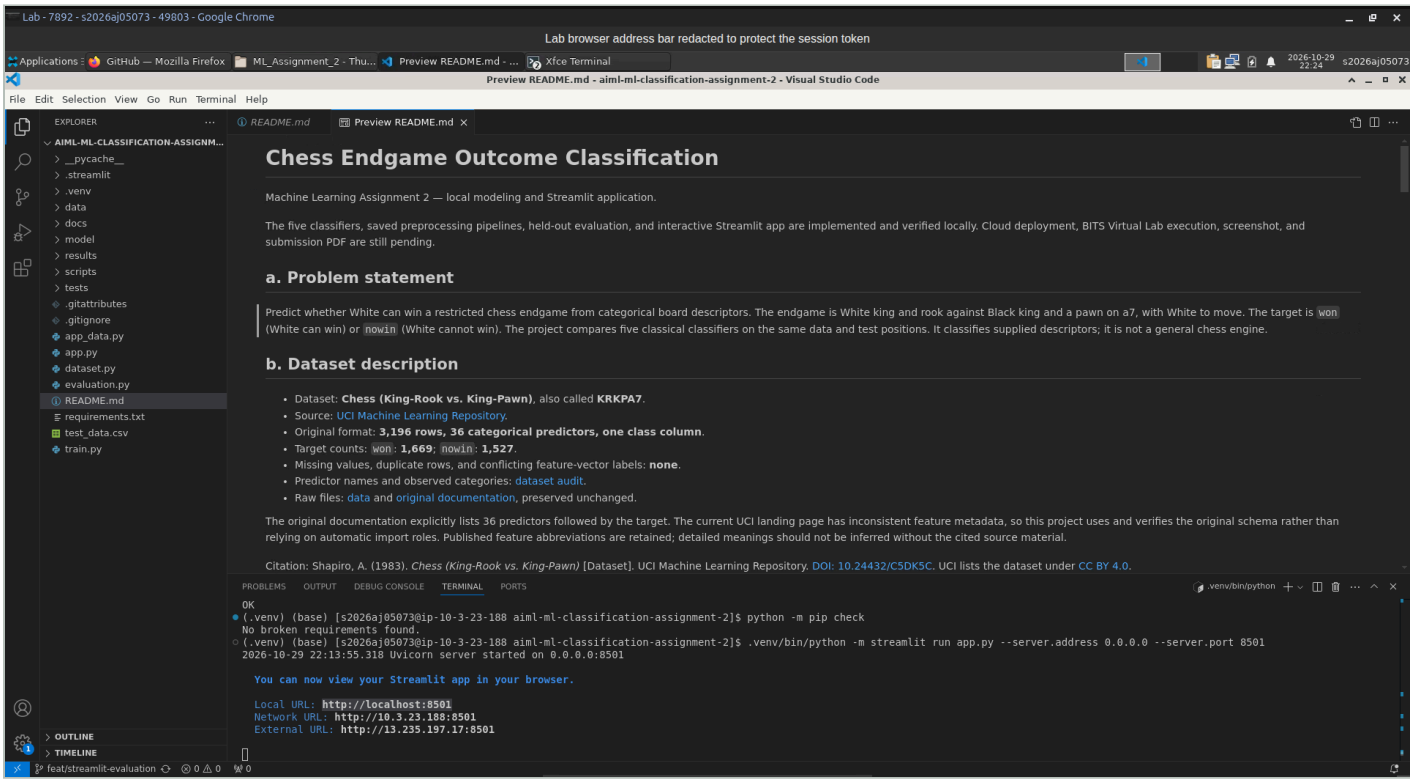
3. BITS Virtual Lab execution evidence

Primary evidence — completed model training



29 October 2026, 22:44. The lab terminal shows completed training, all five models and their metrics, and SVM selection.

Additional execution view — project and Streamlit server

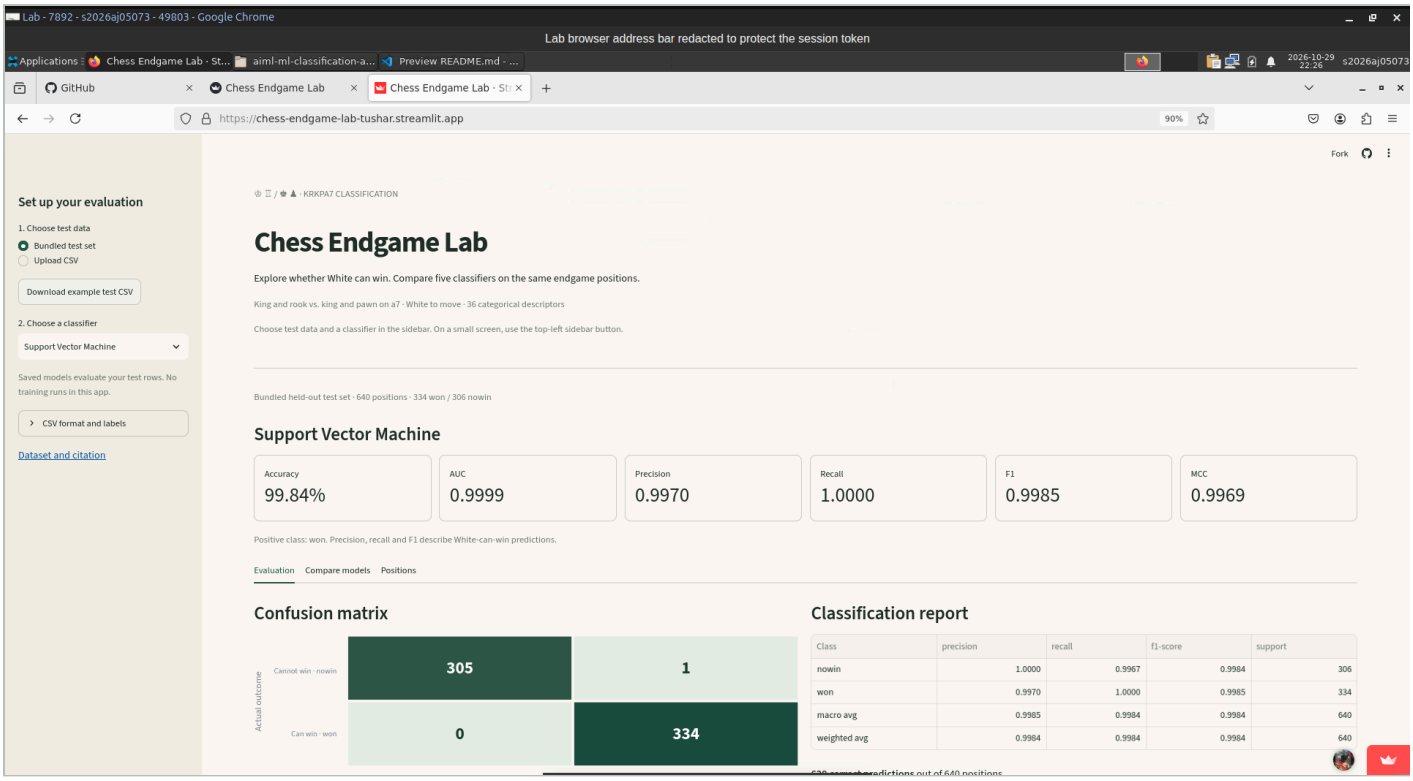


29 October 2026, 22:24. Project files, successful dependency check, and running Streamlit server. Browser address bars are redacted to protect the session token.

Run comparison: lab KNN test F1 is 0.9599; the committed local artifacts report 0.9661. This difference needs reconciliation; the README table reports the local artifacts.

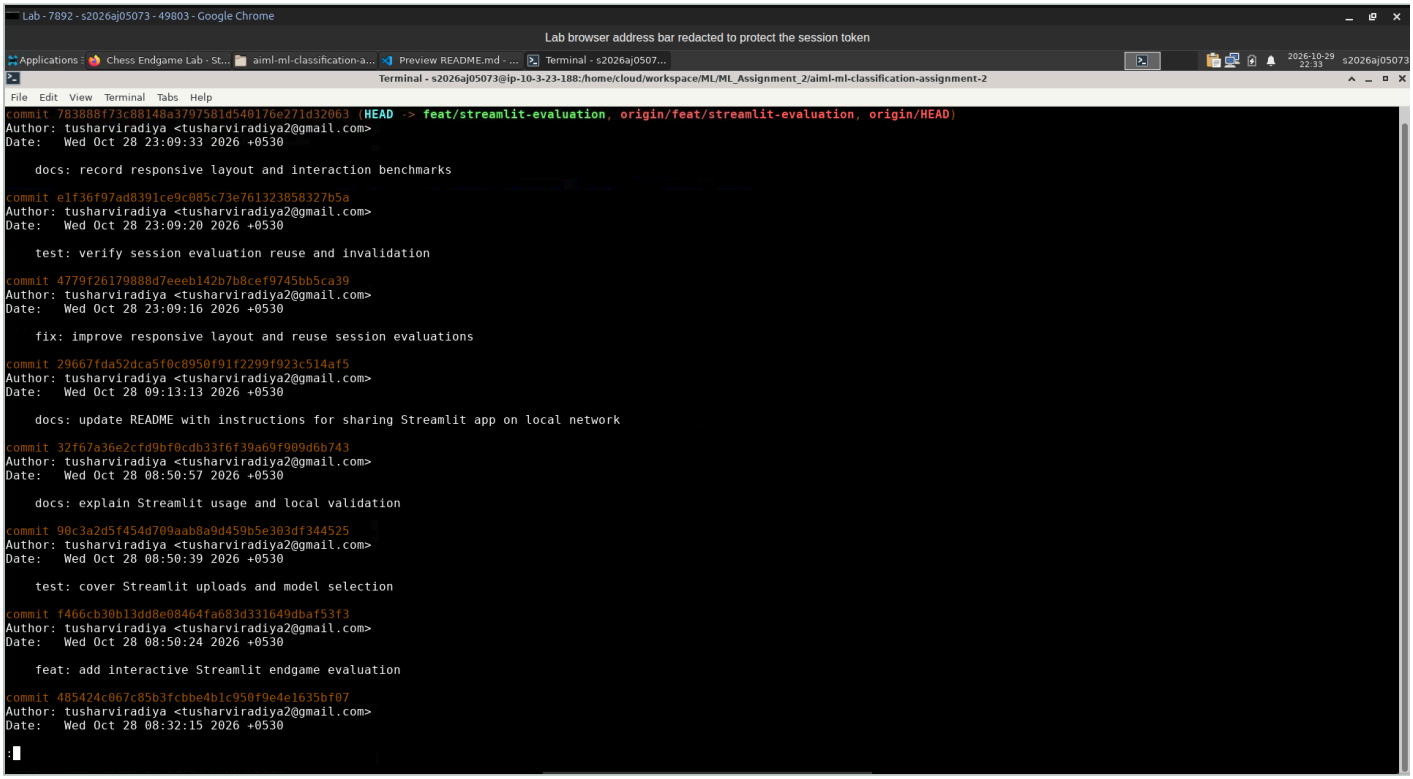
3. Supporting lab-session views

Public Streamlit Community Cloud application



29 October 2026, 22:26. The public cloud app opened from the lab browser. This view demonstrates the cloud interface, not execution of the lab-hosted app.

Repository commit history in the lab terminal



29 October 2026, 22:33. Repository history inspected inside the lab session. Supporting context only; completed training is documented on the preceding page.

All four supplied screenshots are included. Only the outer browser address bars are redacted; the captured execution output is preserved.

4. Project README

Chess Endgame Outcome Classification

Machine Learning Assignment 2 · Live Streamlit app

Five classifiers, saved preprocessing pipelines, held-out results, and a deployed Streamlit app are complete. BITS Virtual Lab execution evidence was captured on 29 October 2026; the KNN reproduction difference below remains to be reconciled.

a. Problem statement

Predict whether White can win with a king and rook against Black's king and pawn on a7, with White to move. The target is `won` or `nowin`. Five classifiers are compared on the same categorical board descriptors and held-out positions.

b. Dataset description

- **UCI Chess (King-Rook vs. King-Pawn), KRKPA7**: dataset source.
- **3,196 rows and 36 categorical predictors**, plus target `outcome`.
- Class counts: **1,669 won**, **1,527 nowin**; no missing values or duplicate rows.
- Original data and documentation are preserved; the dataset audit records schema checks.

The original documentation and raw data confirm 36 predictors despite inconsistent feature metadata on the UCI landing page.

Citation: Shapiro, A. (1983). *Chess (King-Rook vs. King-Pawn)*. UCI Machine Learning Repository. DOI: 10.24432/C5DK5C, CC BY 4.0.

c. GitHub Repository Link

tusharviradiya/aiml-ml-classification-assignment-2

d. Models used

Logistic Regression, Decision Tree, K-Nearest Neighbors, Multinomial Naive Bayes, and RBF Support Vector Machine.

Evaluation procedure

1. Map `nowin` to 0 and `won` to 1. Use a stratified 80/20 split with seed 42: **2,556 training rows and 640 held-out test rows**.
2. Fit one-hot encoding within each pipeline and training fold. The binary indicators require no additional scaling and satisfy Multinomial NB's nonnegative-input requirement.
3. Select parameters by **five-fold stratified training cross-validation F1**; refit each selected pipeline on the training set and save it under `model/`.
4. Select the overall winner by training CV F1, then evaluate all five saved pipelines on the same test set. The app never fits uploaded data.

`train.py` defines the grids; `results/*_cv.csv` stores all candidates. The rare valid category `spcop=t` can be unseen during training, so the encoder ignores unknown categories; separate validation rejects codes outside the audited domains.

Comparison and observations

The following section is generated by training. Review and explain the observations in your own words before submission.

Generated by `python train.py`.

Training rows: 2556. Test rows: 640. Seed: 42. Positive class: won. Settings selected by five-fold training CV F1.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9734	0.9962	0.9789	0.9701	0.9744	0.9468
Decision Tree	0.9969	0.9970	1.0000	0.9940	0.9970	0.9938
KNN	0.9641	0.9939	0.9507	0.9820	0.9661	0.9284
Naive Bayes	0.8609	0.9513	0.8510	0.8892	0.8697	0.7216
Support Vector Machine	0.9984	0.9999	0.9970	1.0000	0.9985	0.9969

Model	CV F1 mean	CV F1 std	Training F1	Test F1
Logistic Regression	0.9685	0.0067	0.9767	0.9744
Decision Tree	0.9952	0.0030	1.0000	0.9970
KNN	0.9494	0.0093	1.0000	0.9661
Naive Bayes	0.8858	0.0139	0.8907	0.8697
Support Vector Machine	0.9966	0.0028	0.9996	0.9985

Selected model: **Support Vector Machine**, based on training CV F1.

Precision, recall and F1 use won as the positive class. AUC uses positive-class probabilities, or decision scores for SVM. MCC uses predicted labels. Precision/recall/F1 are set to zero when their denominator is zero.

SVM scores are decision values, not probabilities. Confusion matrices in evaluation.json use row=true, column=predicted, with order [nowin, won].

These are results for one held-out split of this restricted endgame dataset. They do not establish performance on arbitrary chess positions.

Observations

ML Model Name	Observation about model performance
Logistic Regression	Test F1 0.9744; 7 false wins and 10 missed wins. Training F1 0.9767. An additive linear baseline over encoded categories; it does not explicitly represent feature interactions.
Decision Tree	Test F1 0.9970; 0 false wins and 2 missed wins. Training F1 1.0000. Can learn conjunctions of feature conditions. Compare training and validation results when assessing overfitting.
KNN	Test F1 0.9661; 17 false wins and 6 missed wins. Training F1 1.0000. Distance-weighted training predictions can be perfect because a training row is its own neighbor; cross-validation is the more useful check.
Naive Bayes	Test F1 0.8697; 52 false wins and 37 missed wins. Training F1 0.8907. Models indicators as category counts under conditional independence. Dependencies among board descriptors are a possible limitation, not a proven cause of errors.
Support Vector Machine	Test F1 0.9985; 1 false wins and 0 missed wins. Training F1 0.9996. The RBF kernel permits nonlinear feature interactions. Its measured performance applies to this restricted endgame domain.
Overall winner for this dataset	**Support Vector Machine**, selected using the highest mean training cross-validation F1, before test-set comparison.

A false win means predicting won for a nowin position; a missed win means predicting nowin for a won position. Small score differences on one split do not establish statistical superiority. These observations should be reviewed and developed in the student's own words before submission.

Reproduce the project

Use **Python 3.12 or newer**; the saved run was verified on **Python 3.14.4**. Install the pinned dependencies from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

To reproduce training and validate the project locally or on BITS Virtual Lab:

```
python scripts/audit_dataset.py
python train.py
python -m unittest discover -s tests -v
python -m pip check
```

Training overwrites saved models, splits, results, and the marked README section. Keep the original data, seed, and dependencies unchanged. Versions, source-row indices and checksums are recorded in `results/manifest.json`; selected parameters and detailed evaluations are in `results/evaluation.json`.

Run and use the app

The committed models are ready to load; training is not required to start the app.

```
.venv/bin/python -m streamlit run app.py
```

Open **http://localhost:8501**, or use the deployed app.

1. Select the bundled 640-row test set or upload a labeled CSV, then choose a model.
2. View six metrics, the confusion matrix, classification report and ROC curve.
3. Use **Compare models** for all five models and **Positions** to inspect errors; download comparisons or predictions as CSV.

Uploads must be UTF-8 CSV with all 36 original predictors and target `outcome` (`won`/`nowin`), up to **2 MB and 5,000 rows**. Column order may vary; use `test_data.csv` as the example. Invalid schemas, categories or labels produce readable errors. For single-class uploads, AUC displays **N/A** with an explanation. Uploads stay in memory within the visitor's session; no training occurs in the app.

Validation

The 16 automated checks cover split integrity, saved-model results, CSV validation, all five model choices, uploads without fitting, error filtering and session reuse and isolation. Phone-to-desktop layout checks are documented in the responsiveness report.

Key files

Path	Purpose
`app.py`, `app_data.py`	Streamlit interface, CSV parsing and saved-model inference
`dataset.py`, `evaluation.py`	Schema validation and shared evaluation
`train.py`	Split, cross-validation, training and artifact generation
`data/raw/`, `scripts/audit_dataset.py`	Original dataset and audit
`data/train_data.csv`, `test_data.csv`	Training and held-out labeled data
`model/*.joblib`	Five fitted classifiers with preprocessing
`results/`	Metrics, CV results, parameters, predictions and run manifest
`tests/`	Workflow and Streamlit interaction checks

BITS Virtual Lab execution evidence

Screenshots from **29 October 2026**:

- Completed training — primary evidence: all five classifiers, evaluation metrics and SVM selection.
- Project and Streamlit server: dependency check and server running on port 8501.
- Supporting views: public cloud app and Git history opened within the lab session.

Browser address bars are redacted to protect the session token. The cloud view shows the public deployment; completed lab training is shown in the primary image.

Reproduction issue: lab KNN test F1 is **0.9599**, versus **0.9661** in the committed local artifacts. The cause is unconfirmed. Compare the lab revision, dependencies and selected KNN parameters before claiming exact reproduction. The results table above describes the local artifacts.

Submission

`ML_Assignment_2_Submission.pdf` contains the clickable GitHub and live app links, lab evidence, and full README in the required order. Four screenshots are arranged two per A4 page, with additional views labeled as supporting evidence.

Before submitting, reconcile the KNN difference, explain the observations in your own words, push final changes, and verify both links and the PDF. The assignment allows AI for learning support; understand and verify your implementation. Upload the final PDF to the portal by **8 November 2026, 23:59**.